Version 2.0

January 2026

DATABASE

MODULE 5 - AGGREGATE FUNCTION

Author:

Dr. Ir. Agus Eko Minarno, S. Kom., M. Kom., IPM

Naufal Arkaan

Ismail Dwi Muh. Anugerah

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

INTRODUCTION

OBJECTIVES

1. To help students understand the concept of aggregate functions in SQL.
2. To introduce the use of supporting clauses such as `WHERE`, `GROUP BY`, `HAVING`, and `ORDER BY`.
3. To enable students to analyze academic data using aggregate functions.

MODULE TARGETS

1. Students are able to use basic aggregate functions (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`).
2. Students are able to combine aggregate functions with filtering, grouping, and sorting clauses.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

   رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop or personal computer
3. **Oracle 11g XE:**
   LINK_ORACLE 11G XE
   Oracle apex:
   LINK_ORACLE APEX
   Student Affairs Scheme Link (Table) in the material explanation
   LINK_IMAGE OF THE STUDENT AFFAIRS TABLE
   Link to the Online_Shop schema file for Practice Activity:
   LINK_FILE SCHEMA ONLINE SHOP
4. Strong motivation and commitment to learning

KEYWORDS

Aggregate Function, COUNT, SUM, AVG, GROUP BY, HAVING, ORDER BY

MATERIAL

1. Basic Concepts of Aggregate Functions

1.1 Definition of Aggregate Functions

Aggregate functions are SQL functions used to perform calculations on a set of rows and return a **single summary value**. These functions are often used in data analysis to count the number of records, calculate totals, calculate averages, and find minimum or maximum values.

Aggregate functions **ignore NULL values**, except for `COUNT(*)`, which counts all rows.

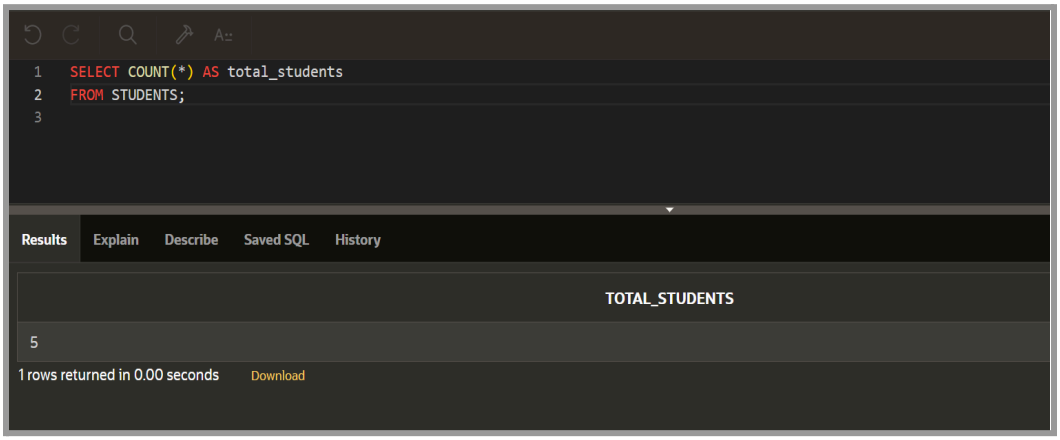
Analogy:

Instead of checking each student individually, a lecturer summarizes the class performance using the total number of students, the average GPA, or the highest score.

2. Basic Aggregate Functions

2.1 COUNT() Function

The `COUNT()` function returns the number of rows in a table.

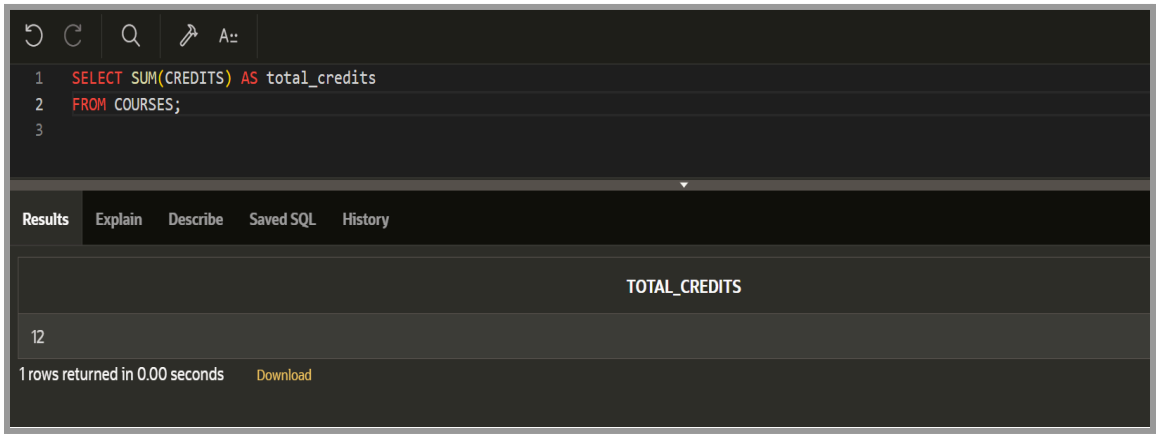


Explanation:
`COUNT(*)` counts all rows in the STUDENTS table.

Analogy:
Counting how many students are registered in the university.

2.2 SUM() Function

The `SUM()` function adds all numeric values in a column.

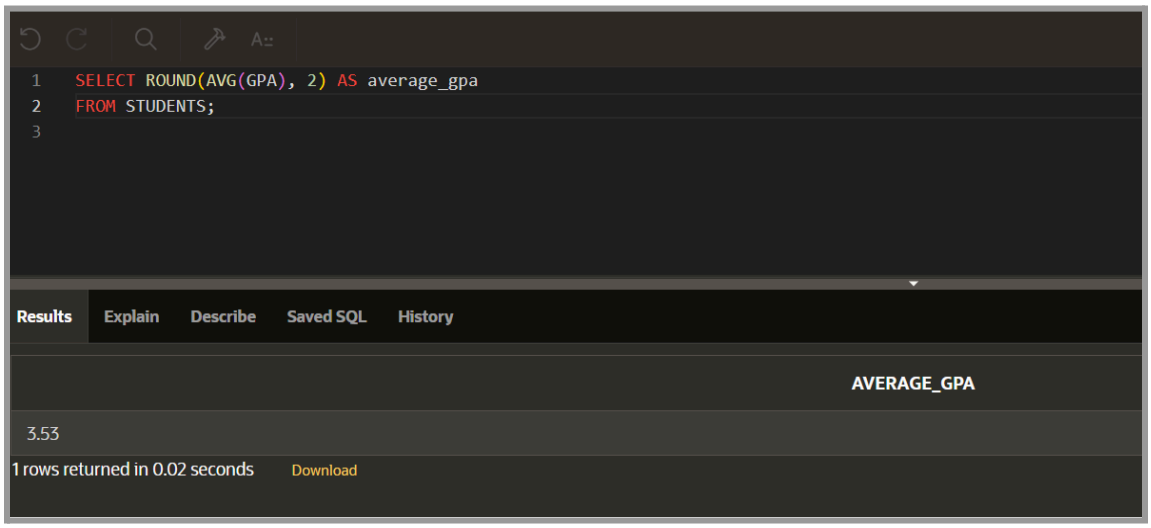


Explanation:
Adds all values in the CREDITS column.

Analogy:
Calculating the total number of credits offered by all courses.

2.3 AVG() Function

The `AVG()` function calculates the average value of a column that is numeric.

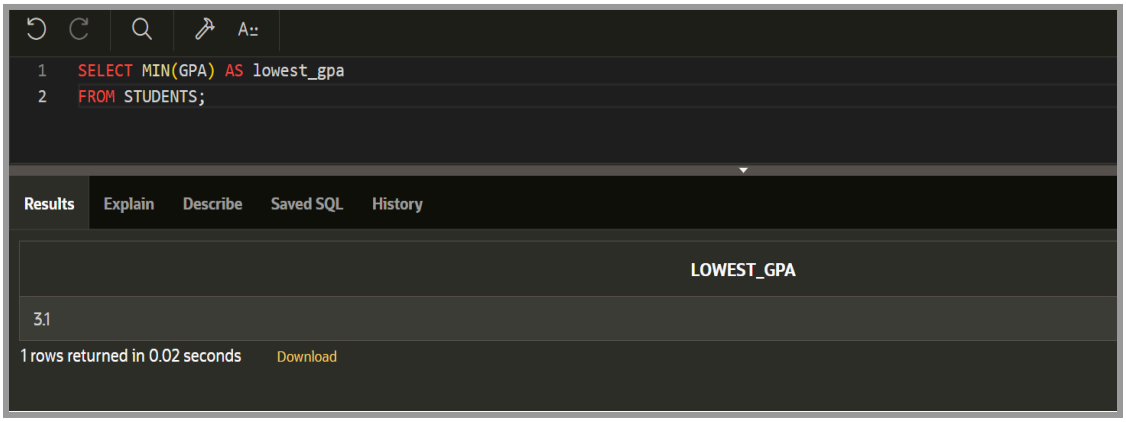


Explanation:
Calculate the average GPA of all students and round it to two decimal places.

Analogy:
Calculate the average GPA of a class.

2.4 MIN() Function

The `MIN()` function returns the smallest value or finds the smallest value in a column.

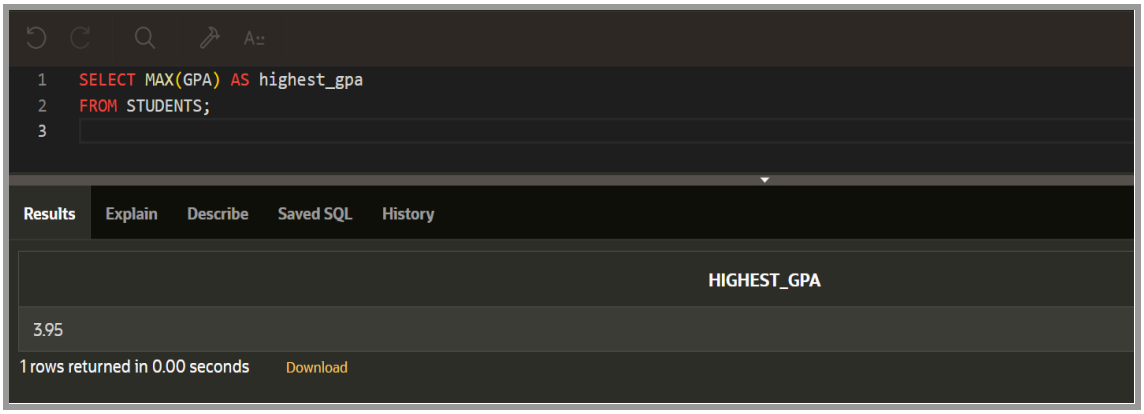


Explanation:
Find the lowest GPA in the STUDENTS table.

Analogy:
Find the lowest test score in a class.

2.5 MAX() Function

The `MAX()` function returns the value or finds the largest value in a column.



Explanation:
Find the highest GPA in the STUDENTS table.

Analogy:
Find the highest exam score in a class.

3. Supporting Clauses for Aggregate Functions

3.1 WHERE Clause

The `WHERE` clause filters rows before aggregate functions are applied, so `WHERE` is included in the data filtering category.

Comparison Operators

Operator	Meaning
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal

Logical Operators

Operator	Meaning
AND	Both conditions must be true
OR	One condition must be true
NOT	Reverses a condition

One example of a comparison operator:

```sql
SELECT COUNT(*) AS students_2024
FROM STUDENTS
WHERE ENROLLMENTYEAR = 2024;
```

Results | Explain | Describe | Saved SQL | History

STUDENTS_2024
2

1 rows returned in 0.02 seconds Download

One example of a logical operator:

```sql
SELECT COUNT(*) AS active_high_gpa
FROM STUDENTS
WHERE GPA >= 3.5 AND STATUS = 'Active';
```

Results | Explain | Describe | Saved SQL | History

ACTIVE_HIGH_GPA
3

1 rows returned in 0.00 seconds Download

Analogy:

Analogy in the comparison operator example:

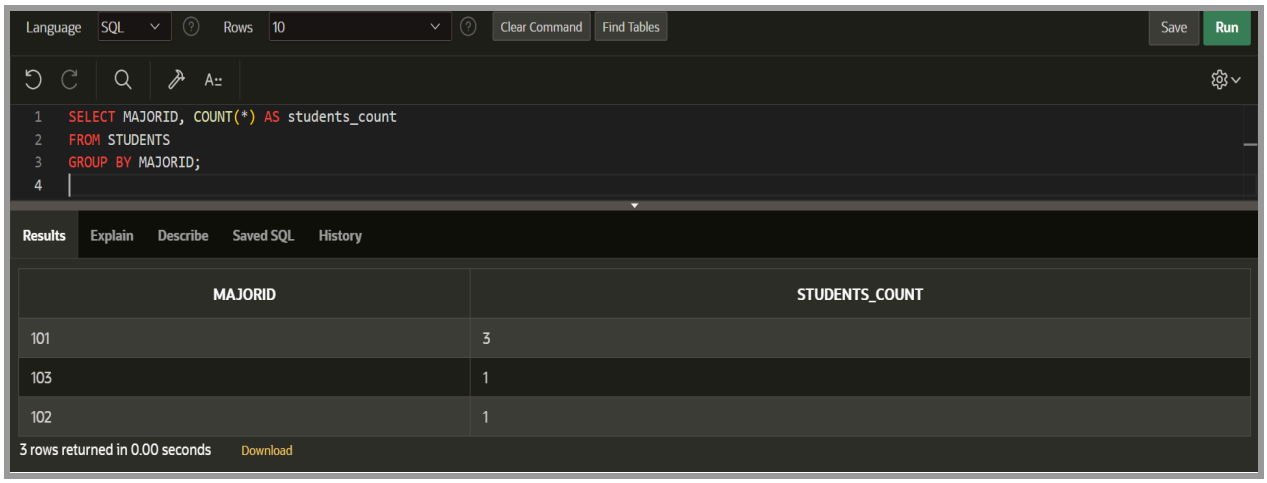
- Filtering the 2024 Academic Year for students

Analogy in the logic operator example:

- Filtering only active students with high GPAs before calculating.

3.2 GROUP BY Clause

The GROUP BY clause groups rows with the same values so that aggregate functions can be applied per group. By using GROUP BY, you can apply aggregate functions (such as SUM, COUNT, AVG, MIN, or MAX) to each group of rows generated by sorting based on a specific column.



Explanation:

Each study program (MAJORID) forms a group, and the `COUNT(*)` function calculates the number of MAJORID students in each study program.

Analogy:

Grouping students by major and counting each group separately.

3.3 HAVING Clause

HAVING is used to determine the groupings to be displayed. Furthermore, having can limit groupings based on Aggregate Function information.

Explanation:

Only majors with more than 2 students are displayed.

Difference from WHERE:

- `WHERE` → filters rows
- `HAVING` → filters groups

Analogy:

After grouping students based on their major ID, only display majors that have many students.

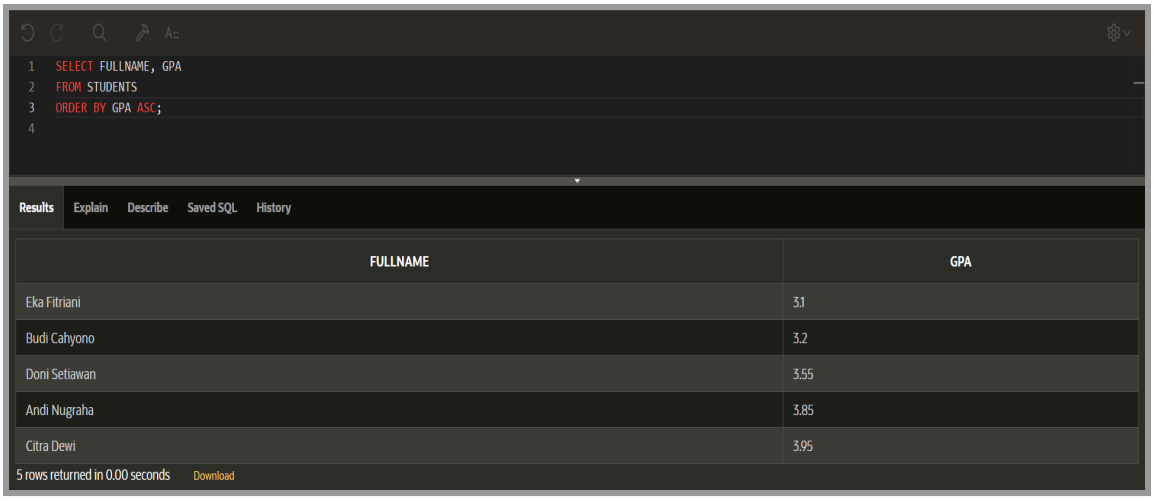
4. Sorting Data (Sorting Query Results)

4.1 Data Sorting Concepts

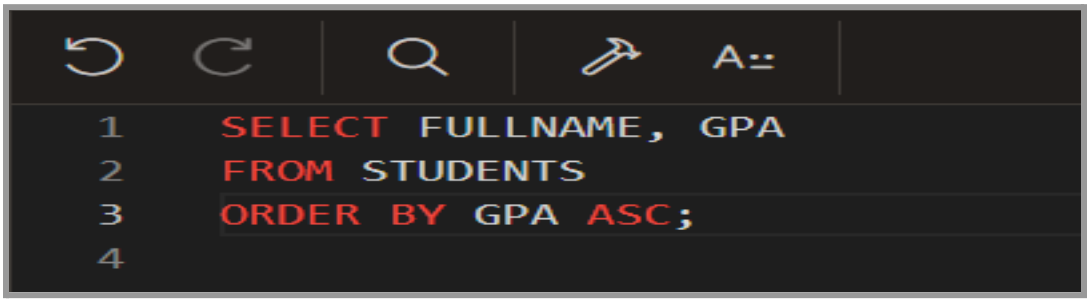
Sorting arranges query results into a specific order using the ORDER BY clause. Sorting is applied **after** data is selected and aggregated.

4.2 Ascending Sorting (ASC)

Ascending order sorts data from smallest to largest.



Query Syntax if you don't see the image above:



Explanation:

The result is sorted from the smallest value to the largest value. Rows with lower GPA appear first, followed by higher GPA values.

Analogy:
Sorting GPA from the lowest to the highest.

4.3 Descending Sorting (DESC)

Descending is sorting data from largest to smallest.

```
1  SELECT FULLNAME, GPA
2  FROM STUDENTS
3  ORDER BY GPA DESC;
```

Results | Explain | Describe | Saved SQL | History

FULLNAME	GPA
Citra Dewi	3.95
Andi Nugraha	3.85
Doni Setiawan	3.55
Budi Cahyono	3.2
Eka Fitriani	3.1

5 rows returned in 0.00 seconds Download

Explanation:
The result is sorted **from the largest value to the smallest value**. Rows with higher GPA appear first, like a ranking list.
Analogy:
Like a leaderboard where **the best score is shown at the top**.

4.4 Sorting Aggregate Function Results

Sorting can also be applied to aggregated results.

```
1  SELECT MAJORID, ROUND(AVG(GPA), 2) AS avg_gpa
2  FROM STUDENTS
3  GROUP BY MAJORID
4  ORDER BY avg_gpa DESC;
```

Results | Explain | Describe | Saved SQL | History

MAJORID	AVG_GPA
102	3.95
103	3.55
101	3.38

3 rows returned in 0.00 seconds Download

Explanation:
Calculates average GPA per major, then sorts majors from highest to lowest average GPA.
Analogy:
Ranking majors based on academic performance.

5. Combining Aggregate Functions in SQL Queries

Aggregate functions are rarely used alone. In real database analysis, they are **combined with filtering, grouping, and sorting clauses** to produce more meaningful results. This section explains how aggregate functions work together with `WHERE`, `GROUP BY`, `HAVING`, and `ORDER BY`.

5.1 Sorting Aggregate Function Results

The `WHERE` clause is used to **filter rows before the aggregation is calculated**. Only rows that satisfy the condition are included in the calculation.

```
1  SELECT COUNT(*) AS students_2024
2  FROM STUDENTS
3  WHERE ENROLLMENTYEAR = 2024;
```

Results | Explain | Describe | Saved SQL | History

STUDENTS_2024
2

1 rows returned in 0.02 seconds Download

Explanation:

- `WHERE ENROLLMENTYEAR = 2024` filters the data first.
- Only students who enrolled in 2024 are counted.
- Aggregation is applied after filtering.

This approach prevents irrelevant rows from affecting the results.

Analogy:

Like selecting only students from one admission year before calculating statistics.

5.2 Aggregate Functions with GROUP BY

`GROUP BY` is used to divide rows into groups so that aggregate functions are calculated per group, not for the entire table.

```
1 SELECT ENROLLMENTYEAR, AVG(GPA) AS avg_gpa_year
2 FROM STUDENTS
3 GROUP BY ENROLLMENTYEAR;
```

Results | Explain | Describe | Saved SQL | History

ENROLLMENTYEAR	AVG_GPA_YEAR
2024	3.9
2022	3.1
2023	3.375

3 rows returned in 0.01 seconds Download

Explanation:

- GROUP BY ENROLLMENTYEAR groups students by enrollment year.
- AVG(GPA) calculates the average GPA for each year.
- The results contain one row per enrollment year.

This query helps us compare the academic performance of different student cohorts.

Analogy:

Such as comparing and analyzing the average performance of each class per year, for example: the class of 2022 vs. the class of 2023 vs. the class of 2024.

5.3 Aggregate Functions with HAVING

HAVING is used to filter groups after the aggregation process is done. Its function is similar to WHERE, but it's applied at the group level. For example, selecting only majors whose average GPA exceeds a certain threshold.

```
1 SELECT MAJORID, AVG(GPA) AS avg_gpa
2 FROM STUDENTS
3 GROUP BY MAJORID
4 HAVING AVG(GPA) >= 3.2;
```

Results | Explain | Describe | Saved SQL | History

MAJORID	AVG_GPA
101	3.383333333333333333333333333333333333
103	3.55
102	3.95

3 rows returned in 0.01 seconds Download

Explanation:

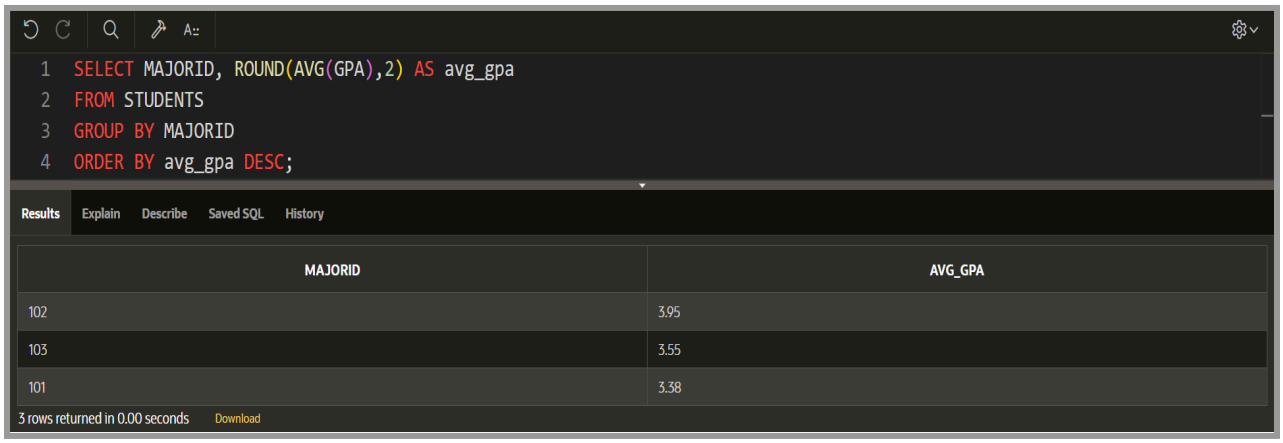
- GROUP BY MAJORID groups students by major
- AVG(GPA) calculates the average GPA per major
- HAVING AVG(GPA) >= 3.2 only retains majors with high GPAs

WHERE cannot be used in this case because the condition is based on aggregate results.

Analogy:
Similar to analyzing each major first, the next step is to only display majors with good academic achievements.

5.4 Aggregate Functions with ORDER BY

`ORDER BY` is useful for sorting the final aggregation results, either in ascending (ASC) or descending (DESC) order.



```
1  SELECT MAJORID, ROUND(AVG(GPA),2) AS avg_gpa
2  FROM STUDENTS
3  GROUP BY MAJORID
4  ORDER BY avg_gpa DESC;
```

Results | Explain | Describe | Saved SQL | History

MAJORID	AVG_GPA
102	3.95
103	3.55
101	3.38

3 rows returned in 0.00 seconds Download

Explanation:

- This query calculates the average GPA per study program.
- `ORDER BY avg_gpa DESC` is used to sort the results from the highest to the lowest average GPA (DESC).
- This makes it easier to analyze the study programs with the best performance.

Analogy:
Like making a ranked list where the highest scores are displayed first, followed by the lowest scores.

6. Additional Functions (Introduction)

Additional aggregate functions are used to perform more specific data analysis. These functions are not always necessary in basic SQL operations, but are useful in situations where faster and more sophisticated data summarization is required. The functions discussed in this section include `LISTAGG()`, `MEDIAN()`, `RANK()`, and `CASE WHEN` in aggregate operations.

6.1 LISTAGG() Function

The `LISTAGG()` function is used to combine multiple text values from different rows into a single string, usually separated by commas or spaces. This function is useful for displaying grouped text data in a concise form.

```
SELECT MAJORID,
       LISTAGG(FULLNAME, ', ')
       WITHIN GROUP (ORDER BY FULLNAME) AS student_list
FROM STUDENTS
GROUP BY MAJORID;
```

Results | Explain | Describe | Saved SQL | History

MAJORID	STUDENT_LIST
101	Andi Nugraha, Budi Cahyono, Eka Fitriani
102	Citra Dewi
103	Doni Setiawan

Explanation:

- Students are grouped based on MAJORID
- `LISTAGG()` combines all student names in the same major
- The result displays one row per major containing a list of names

Analogy:
Like writing a class attendance list in one line, for example: “Anna, Budi, Clara, David”.

6.2 MEDIAN() Function

`MEDIAN()` returns the middle value in a sorted set of numeric data. This function is useful when the average value can be affected by extreme values (outliers).

```
SELECT MAJORID,
       MEDIAN(GPA) AS median_gpa
FROM STUDENTS
GROUP BY MAJORID;
```

Results | Explain | Describe | Saved SQL | History

MAJORID	MEDIAN_GPA
101	3.2
102	3.95
103	3.55

Explanation:

- Students are grouped by major
- `MEDIAN(GPA)` finds the **middle GPA value** in each group
- The result represents the **central performance level** of each major

Analogy:
If GPAs are sorted like
2.8, 3.0, 3.2, 3.5, 3.7
The **middle value (3.2)** is the median.

6.3 RANK() Function

RANK( ) is a window function used to assign a rank number to rows based on a specific order. If two rows have the same value, they will both receive the same rank.

```
SELECT NIM, FULLNAME, GPA,
       RANK() OVER (ORDER BY GPA DESC) AS gpa_rank
FROM STUDENTS;
```

Results | Explain | Describe | Saved SQL | History

NIM	FULLNAME	GPA	GPA_RANK
2024002	Citra Dewi	3.95	1
2024001	Andi Nugraha	3.85	2
2023020	Doni Setiawan	3.55	3
2023015	Budi Cahyono	3.2	4
2022030	Eka Fitriani	3.1	5

Explanation:

- Rows are ordered by GPA from highest to lowest
- The highest GPA receives **rank 1**
- Students with the same GPA receive the same rank

Analogy:

Like a competition ranking table where two students can share first place if their scores are equal.

6.4 Using CASE WHEN in Aggregate Functions

CASE WHEN is used to apply logic within aggregate queries. This feature allows calculations or summations to be performed only on rows that meet certain conditions.

```
SELECT
    SUM(CASE WHEN GPA >= 3.5 THEN 1 ELSE 0 END) AS high_gpa_students,
    SUM(CASE WHEN GPA < 3.5 THEN 1 ELSE 0 END) AS regular_gpa_students
FROM STUDENTS;
```

Results | Explain | Describe | Saved SQL | History

HIGH_GPA_STUDENTS	REGULAR_GPA_STUDENTS
3	2

Explanation:

- Each condition produces 1 (true) or 0 (false)
- SUM( ) counts how many rows satisfy the condition
- Two different categories are counted in **one query**

Analogy:

Like counting

“How many students are **above 3.5 GPA** and how many are **below 3.5 GPA**” without running two separate queries.

SUMMARY TABLE of MATERIALS

Query	How to Read in Indonesian (Brief Analogy)
`SELECT COUNT(*) FROM STUDENTS;`	Pilih dan hitung semua baris data dari tabel STUDENTS.
`SELECT SUM(CREDITS) FROM COURSES;`	Pilih dan jumlahkan semua nilai kolom CREDITS dari tabel COURSES.
`SELECT AVG(GPA) FROM STUDENTS;`	Pilih dan hitung nilai rata-rata dari kolom GPA pada tabel STUDENTS.
`SELECT MIN(GPA) FROM STUDENTS;`	Pilih dan ambil nilai GPA yang paling kecil dari tabel STUDENTS.
`SELECT MAX(GPA) FROM STUDENTS;`	Pilih dan ambil nilai GPA yang paling besar dari tabel STUDENTS.
`SELECT COUNT(*) FROM STUDENTS WHERE ENROLLMENTYEAR = 2024;`	Pilih dan hitung jumlah mahasiswa dari tabel STUDENTS yang tahun masuknya sama dengan 2024.
`SELECT COUNT(*) FROM STUDENTS WHERE GPA >= 3.5 AND STATUS = 'Active';`	Pilih dan hitung mahasiswa yang IPK-nya 3.5 atau lebih dan statusnya Active.
`SELECT MAJORID, COUNT(*) FROM STUDENTS GROUP BY MAJORID;`	Kelompokkan mahasiswa berdasarkan MAJORID, lalu hitung jumlah mahasiswa di setiap major.
`SELECT MAJORID, AVG(GPA) FROM STUDENTS GROUP BY MAJORID;`	Kelompokkan mahasiswa berdasarkan MAJORID, lalu hitung rata-rata GPA di setiap major.
`SELECT MAJORID, COUNT(*) FROM STUDENTS GROUP BY MAJORID HAVING COUNT(*) > 2;`	Setelah dikelompokkan per MAJORID, tampilkan hanya major yang jumlah mahasiswanya lebih dari dua.
`SELECT FULLNAME, GPA FROM STUDENTS ORDER BY GPA ASC;`	Tampilkan nama dan GPA mahasiswa, lalu urutkan dari GPA terkecil ke terbesar.
`SELECT FULLNAME, GPA FROM STUDENTS ORDER BY GPA DESC;`	Tampilkan nama dan GPA mahasiswa, lalu urutkan dari GPA terbesar ke terkecil.
`SELECT MAJORID, AVG(GPA) FROM`	Kelompokkan mahasiswa per MAJORID,

`STUDENTS GROUP BY MAJORID ORDER BY AVG(GPA) DESC;`	hitung rata-rata GPA tiap major, lalu urutkan dari rata-rata tertinggi ke terendah.
`SELECT MAJORID, LISTAGG(FULLNAME, ', ') WITHIN GROUP (ORDER BY FULLNAME) FROM STUDENTS GROUP BY MAJORID;`	Kelompokkan mahasiswa per MAJORID, lalu gabungkan semua nama mahasiswa dalam satu baris, dipisahkan koma.
`SELECT MAJORID, MEDIAN(GPA) FROM STUDENTS GROUP BY MAJORID;`	Kelompokkan mahasiswa per MAJORID, lalu ambil nilai GPA tengah dari setiap kelompok.
`SELECT NIM, FULLNAME, GPA, RANK() OVER (ORDER BY GPA DESC) FROM STUDENTS;`	Tampilkan data mahasiswa dan beri peringkat berdasarkan GPA dari yang tertinggi ke yang terendah.
`SELECT SUM(CASE WHEN GPA >= 3.5 THEN 1 ELSE 0 END) FROM STUDENTS;`	Hitung jumlah mahasiswa yang GPA-nya lebih besar atau sama dengan 3.5.

PRACTICE ACTIVITY

For this Practice Activity, we will continue with the online shop scheme that we worked on in Module 4. Please pay attention and follow these steps:

Case Story — New Customers and New Transactions
The online shop has recorded several new customers and new purchase transactions.
You are asked to insert the following data into the system.

Students must use the given ID values so the next tasks work correctly.

1. **Step 1 - Insert New Customers (CUSTOMERS)**
   Add the following new customers:

CustomerID	Name	Email
2002	Lina Mahendra	lina@mail.com
2003	Arif Nugroho	arif@mail.com

Add this query to Oracle 11g xe:

```
INSERT INTO CUSTOMERS (CustomerID, CustomerName, Email)
VALUES (2002, 'Lina Mahendra', 'lina@mail.com');

INSERT INTO CUSTOMERS (CustomerID, CustomerName, Email)
VALUES (2003, 'Arif Nugroho', 'arif@mail.com');
```

2. **Step 2 - Insert New Orders (ORDERS)**
   Add two new orders made by the new customers:

OrderID	CustomerID	TotalAmount
11	2002	700000
12	2003	850000

```
INSERT INTO ORDERS (OrderID, CustomerID, TotalAmount)
VALUES (11, 2002, 700000);

INSERT INTO ORDERS (OrderID, CustomerID, TotalAmount)
VALUES (12, 2003, 850000);
```

3. **Step 3 - Insert Order Details (ORDER_DETAILS)**
   Add product purchase details for each order:

OrderDetailID	OrderID	ProductID	Qty	Subtotal
3	11	A001	1	150000
4	11	A002	1	550000
5	12	A001	2	300000
6	12	A002	1	550000

```
INSERT INTO ORDER_DETAILS (OrderDetailID, OrderID, ProductID, Quantity, Subtotal)
VALUES (3, 11, 'A001', 1, 150000);

INSERT INTO ORDER_DETAILS (OrderDetailID, OrderID, ProductID, Quantity, Subtotal)
VALUES (4, 11, 'A002', 1, 550000);

INSERT INTO ORDER_DETAILS (OrderDetailID, OrderID, ProductID, Quantity, Subtotal)
VALUES (5, 12, 'A001', 2, 300000);

INSERT INTO ORDER_DETAILS (OrderDetailID, OrderID, ProductID, Quantity, Subtotal)
VALUES (6, 12, 'A002', 1, 550000);
```

After completing this step, we proceed to the dock query stage using the aggregate function.

LEARNING RESOURCES AND TUTORIALS

LINK_W3SCHOOLS_SQL **(Recommendations)**
LINK_ORACLE_DOCS **(Recommendations)**
LINK_LEARNITWEB

QUERY DOCK

This dock query continues from the practice activity stage and uses the same scheme. In this section, students are not given a skeleton query. They only read the case requirements/view the output table format, **then compose the SQL query themselves**.

1. **Question 1 - Case Study (Story-Based)**

   Case - Customer Spending Report

   A store manager wants to analyze customer behavior. He wants to know **how much total money each customer has spent** based on all orders recorded in the system.

   Task:
   - Create a query to display the **CustomerID** and **Total Spending** for each customer. Concept

   Hint:
   - Use aggregate functions and group the data by customer.

2. **Question 2 - Case Study (Story-Based)**

   Case — Product Sales Performance

   The finance team wants to see **which products contribute the highest revenue**. They want to calculate **total revenue for each product** based on all order transactions.

   Task:
   - Create a query to display **ProductID** and **Total Revenue per product**, then sort the results from **largest to smallest**.

   Hint:
   - Use the subtotal value summary and sort the analysis results.

3. **Question 3 — Interpretation of Output Tables**

   Instruction:

   Take a look at the following output table:

ProductID	total_sold
A002	3
A001	4

Task:

- Create a query that can generate the table output as shown above.

Mindset being tested:

- understanding **`GROUP BY`**
- understanding **`SUM / COUNT`** in transaction details
- ability to translate **table format → query**

4. **Question 4 — Interpretation of Output Tables**

Instruction:

Take a look at the following output table:

CustomerID	order_count
1001	1
2003	1
2002	1

Task:

- Create a query that can generate the table output as shown above.

Implicitly directed concept:

- **`COUNT`** per customer
- **`GROUP BY`** customer

ASSIGNMENT

ASSIGNMENT 1

For this first assignment, use the scheme according to the theme you chose in the previous module (the scheme that has been filled in the spreadsheet file).

Follow these steps:

1. **Step 1 — Add Minimal Additional Data (Data Support for Aggregate)**
   Instructions:
   - Add at least 4 new data entries to the table in each schema (according to the theme: supermarket, office, campus, store, etc.).

   Data requirements:
   - vary (not all values are the same), and
   - allow the use of aggregate functions, for example:
     - there are numerical values to be summed/averaged,
     - there are category columns to be grouped (`GROUP BY`),
     - there are different values that can be compared (`MIN/MAX`).

   There is no need to follow a specific structure — feel free to adapt to the context of the scheme.
   The purpose of this step is simply to ensure that there is sufficient data for analysis.

2. **Step 2 — Perform Aggregation & Generate Insight**
   From the data added in Step 1, do the following:
   - **At least 3 aggregate functions (choose 3 aggregate processes that have been studied in this module), for example:**
     - `COUNT()` → counts the number of data
     - `GROUP BY` → summary by category
     - `ORDER BY` → sequence analysis / ranking

   Students are free to determine the context of the analysis according to the theme. After performing the three aggregate functions, please present your findings to the assistant regarding insights/interpretations, for example: "By using sorting in descending order, we can easily see which product contributes the most sales."

ASSIGNMENT 2 (LIVE DEMO)

- Students will be asked to create a simple table according to the assistant's instructions during the practicum.
- After the table has been successfully created, students are required to input some data (a small amount of data, not excessive) according to the example provided by the assistant.
- The maximum time allowed for each student to complete the assignment is 20 minutes after the instructions are given.
- If the students failed to do the instructions, the assistant will provide several questions related to the **use of Aggregate Functions (`COUNT`, `SUM`, `AVG`, `GROUP BY`, `HAVING`, `ORDER BY`, etc.)**.

Important Notes:

- Students are required to study the module material in advance before the practicum begins.
- Opening notes, books, other devices, or any reference materials during the live demo is not permitted.
- **For students who cannot complete all of the demo instructions, additional questions will be given to increase their score. There are four questions covering basic aggregate and data sorting material. Please study this material as well!**

PRACTICUM GRADING RUBRIC

Assessment Aspects	Points
Query Dock	**Total 10%**
Assignment 1	**Total 20%**
Add Data	5%
Perform Aggregate Query	5%
Explain three aggregate	10%
Assignment 2 (Live Demo)	**Total 70%**
Create Table + Add Data	5%
Basic Aggregate	10%
Sorting Data	15%
Combining Aggregate	20%
Additional Function	20%

Important Notes on Practicum Implementation

During **Query Dock** week, students are encouraged to **ask questions** about module 5 material that they do not yet understand. This session not only tests syntax, but also helps students deepen their understanding of SQL concepts through discussions with assistants.

The **Query Dock score has a small percentage**, so the main focus of the assessment remains on **understanding the material** and the student's ability to explain the logic of the query.

For each assessment item on the rubric:

- **100 → Correct and confident answer / clear explanation**
- **50 → Partially correct answer / still uncertain**
- **0 → Unable to answer / does not follow instructions**

With this scheme, assessment is carried out objectively and consistently according to the level of student mastery.